التجريد والدوال الافتراضية (Abstraction & Virtual Functions)

1 المؤشرات والكائنات — Pointers & Objects

الفكرة الأساسية:

المؤشر متغير يحمل **عنوان** موقع في الذاكرة بدلاً من القيمة نفسها. الكائنات تتصرف مثل أي متغير عادي — يمكن صنع مؤشر يشير إليها بنفس الطريقة

```cpp
class employee
{
    int code;
    char name[20];
public:
    void getdata();
    void display();
};

employee *abc;          // مؤشر من نوع employee
employee obj;
abc = &obj;             // abc يشير للكائن obj
abc->getdata();         // الوصول للدوال عبر المؤشر بـ ->
```

> 📌 القاعدة: مع الكائن المباشر نستخدم النقطة `.` ، ومع المؤشر للكائن نستخدم السهم `->`

2 المؤشر this

`this` مؤشر **خفي تلقائي** يُرسَل لكل دالة عضو عند استدعائها، ويشير للكائن الذي استدعى الدالة. يُحتاج إليه خصوصاً عندما يكون اسم الوسيط **نفس اسم** عضو الكلاس

```cpp
class stud
{
    int a;
public:
    void set(int a)
    {
        this->a = a;    // this->a هو عضو الكلاس
                        //          a  هو الوسيط (argument)
    }
    void show() { cout << a; }
};

int main()
{
    stud S1;
    S1.set(5);
    S1.show();          // 5 :المخرج
}
```

> ⚠ فخ امتحاني: لو كتبنا `a = a` بدون `this->` داخل الدالة، الوسيط سيُسنَد لنفسه وعضو الكلاس لا يتغير!
> الحل الصحيح هو `this->a = a`

3 مؤشر الأساس يشير للمشتق — Pointer to Derived Class

هذه **نقطة الانطلاق الحقيقية** للـ Polymorphism. مؤشر من نوع الكلاس الأساسي **يمكنه** الإشارة لكائنات الكلاسات المشتقة — لكن ليس العكس

```cpp
class base   { /* ... */ };
class derived : public base { /* ... */ };

int main()
{
    base    *ptr;           // مؤشر من نوع الأساس
    derived  obj;

    ptr = &obj;             // ✅ صحيح — الأساس يشير للمشتق
}
```

وماذا عن العكس؟

```cpp
int main()
{
    base    obja;
    derived *ptr;

    ptr = &obja;                    // ❌ خطأ مباشرة
    ptr = (derived*) &obja;         // ✅ يعمل لكن يحتاج Explicit Casting
}
```

الحالة	الحكم
مؤشر أساس = عنوان كائن مشتق	✅ مباشرة بدون casting
مؤشر مشتق = عنوان كائن أساس	❌ يحتاج `(*derived)` صراحةً

4 الدوال الافتراضية — Virtual Functions

ما هدفها؟

تمكين البرنامج من تحديد **أيّ نسخة** من الدالة يُنفَّذ (الأب أم الابن) في وقت التشغيل عبر المؤشر — وهذا يُسمى Dynamic Binding أو Late Binding

الصيغة:

```cpp
class ClassName
{
public:
    virtual (وسائط)نوع_الإرجاع اسم_الدالة;   // ✅ virtual في الإعلان فقط
};
```

قواعد الدوال الافتراضية — احفظها كلها للـ True/False

القاعدة	صحيح أم خطأ؟
`virtual` تُكتب في الإعلان داخل الكلاس فقط	✅ صحيح
`virtual` تُكتب أيضاً في التعريف خارج الكلاس	❌ خطأ — تُحذف من التعريف
يمكن للدالة الافتراضية أن تكون `static`	❌ خطأ — ممنوع تماماً
يمكن للبنّاء (Constructor) أن يكون `virtual`	❌ خطأ — ممنوع
يمكن للمدمِّر (Destructor) أن يكون `virtual`	✅ صحيح ومفيد
يمكن تغيير نوع الإرجاع في الكلاس المشتق	❌ خطأ — نفس النوع إلزامي
الدوال غير الأعضاء يمكن تعريفها `virtual`	❌ خطأ — فقط دوال الأعضاء

فخ تغيير نوع الإرجاع:

```cpp
class base {
public:
    virtual int sum(int x, int y);     // نوع الإرجاع int
};

class derived : public base {
public:
    virtual float sum(int x, int y);  // ❌ خطأ — غيّر النوع لـ float
};
```

الصحيح: كلا الكلاسين يستخدمان نفس نوع الإرجاع

5 Static Binding ضد Dynamic Binding

هذا الفرق **جوهر سؤال Tracing** في أسئلة المؤشرات والوراثة

العنصر	Static Binding (مبكر)	Dynamic Binding (متأخر)
وقت التحديد	وقت التصريف (Compile Time)	وقت التشغيل (Run Time)
كيف يتم	دوال عادية (بدون virtual)	دوال `virtual`
مع المؤشر	يستدعي دالة **الأساس** دائماً	يستدعي دالة **الكائن الفعلي**
الكفاءة	أسرع	أبطأ قليلاً لكن أذكى

المثال الفاصل — Static vs Dynamic:

```cpp
// ——— برنامج 1: بدون virtual (Static Binding) ———
class student {
public:
    void getdata();      // دالة عادية
    void display();
};
class academic : public student {
public:
    void getdata();
    void display();
};

int main()
{
    student  *ptr;
    academic  obj;
    ptr = &obj;
    ptr->getdata();    // ❌ يستدعي getdata() الأساس (student)
    ptr->display();    // ❌ يستدعي display() الأساس (student)
}
```

```cpp
// ────── برنامج 2: مع virtual (Dynamic Binding) ──────
class student {
public:
    virtual void getdata();   // virtual
    virtual void display();
};
class academic : public student {
public:
    void getdata();
    void display();
};

int main()
{
    student  *ptr;
    academic  obj;
    ptr = &obj;
    ptr->getdata();    // ✅ يستدعي getdata() المشتق (academic)
    ptr->display();    // ✅ يستدعي display() المشتق (academic)
}
```

تمرين Tracing — مصفوفة مؤشرات:

```cpp
class baseA {
public:
    virtual void display() { cout << "One\n"; }
};
class derivedB : public baseA {
public:
    virtual void display() { cout << "Two\n"; }
};
class derivedC : public derivedB {
public:
    virtual void display() { cout << "Three\n"; }
};

int main()
{
    baseA    obja;
    derivedB objb;
    derivedC objc;

    baseA *ptr[3];
    ptr[0] = &obja;
    ptr[1] = &objb;
    ptr[2] = &objc;

    for(int i = 0; i <= 2; i++)
        ptr[i]->display();
}
```

ptr[i]	يشير لـ	المخرج
ptr[0]	obja (baseA)	One
ptr[1]	objb (derivedB)	Two
ptr[2]	objc (derivedC)	Three

المخرج الكامل: One ثم Two ثم Three

6 تقطيع الكائن — Object Slicing

عند تعيين كائن **مشتق** لمتغير من نوع **الأساس** مباشرةً (بدون مؤشر)، الأعضاء الإضافية للمشتق **تُقطَع وتُفقَد**

```cpp
class Base    { int x, y; };
class Derived : public Base { int z, w; };

int main()
{
    Derived d;
    Base b = d;     // ✅ يعمل لكن z وw يُفقدان — Object Slicing!
}
```

المشكلة عملياً:

```cpp
void somefunc(Base obj)          // Slicing يأخذ بالقيمة ← يحدث
{
    obj.display();
}

int main()
{
    Base    b(33);
    Derived d(45, 54);
    somefunc(b);                 // مخرج: I am Base, i=33
    somefunc(d);                 // مخرج: I am Base, i=45  ❌ (طُبع كأساس!)
}
```

الحل — استخدام Reference أو Pointer:

```cpp
void somefunc(Base &obj)         // Slicing مرجع — لا
{
    obj.display();
}
// أو
void somefunc(Base *obj)         // Slicing مؤشر — لا
{
    obj->display();
}
```

طريقة التمرير	Object Slicing؟
بالقيمة `Base obj`	✅ يحدث — الأعضاء الزائدة تُفقَد
بالمرجع `Base &obj`	❌ لا يحدث
بالمؤشر `Base *obj`	❌ لا يحدث

---

7 تجاوز الدوال — Function Overriding

عندما يُعيد الكلاس المشتق تعريف دالة **بنفس الاسم ونفس الوسائط** الموجودة في الأساس — هذا يُسمى **Function Overriding**. الكلاس المشتق يُخفي نسخة الأساس

```cpp
class A
{
public:
    void display() { cout << "Class A\n"; }
};

class B : public A
{
public:
    void display() { cout << "Class B\n"; }  // Override
};

int main()
{
    B obj;
    obj.display();        // B الخاص بـ display() يستدعي
    obj.A::display();     // :: صراحةً نستخدم A لاستدعاء نسخة
}
```

المخرج:

```text
Class B
Class A
```

> 📌 الفرق بين Overriding وOverloading: الـ Overloading داخل نفس الكلاس بوسائط مختلفة، أما Overriding فهو بين أساس ومشتق بنفس الوسائط

8 الكلاس المجرد — Abstract Class

الكلاس المجرد يحتوي على **دالة افتراضية نقية (Pure Virtual Function)** واحدة على الأقل — صيغتها تنتهي بـ `= 0`

خصائصه الأربع — للحفظ:

- ❌ لا يمكن إنشاء **كائن** منه مباشرة — لكن يمكن إنشاء **مؤشر أو مرجع** منه
- ✅ يمكنه امتلاك **دوال عادية ومتغيرات** إلى جانب الدوال النقية
- ✅ يُستخدم أساساً للـ **Upcasting** — أي مؤشره يشير لكائنات المشتقات
- ✅ الكلاس المشتق منه **يجب** أن يُنفِّذ جميع الدوال النقية وإلا أصبح هو أيضاً مجرداً

```cpp
class Base                          // كلاس مجرد
{
public:
    virtual void show() = 0;        // دالة نقية
};

class Derived : public Base
{
public:
    void show()                     // ✅ تنفيذ الدالة النقية
    {
        cout << "Implementation in Derived" << endl;
    }
};

int main()
{
    Base obj;           // ❌ Compile Error — لا يمكن إنشاء كائن مجرد
    Base *b;            // ✅ مسموح — مؤشر من نوع المجرد
    Derived d;          // ✅ مسموح — d ليس مجرداً
    b = &d;
    b->show();          // ✅ Dynamic Binding — يستدعي show() المشتق
}
```

المخرج: `Implementation in Derived`

> ⚠️ فخ امتحاني: السطر `Base obj` يُعطي Compile Error لأن VTABLE الكلاس المجرد غير مكتملة (الخانة المخصصة للدالة النقية فارغة بدون عنوان) — لذا المُصرّف يرفض إنشاء الكائن

📊 جدول المقارنة الشامل — Virtual vs Pure Virtual vs Override

العنصر	دالة عادية	`virtual`	`virtual = 0` (نقية)
الجسم (Body)	✅ موجود	✅ موجود	❌ لا جسم
يجعل الكلاس مجرداً؟	❌	❌	✅
Binding	Static (مبكر)	Dynamic (متأخر)	Dynamic (متأخر)
إلزامية التنفيذ بالمشتق	اختيارية	اختيارية	**إلزامية**
إنشاء كائن من الكلاس	✅	✅	❌

> أيٌّ مما يلي مسموح للدوال الافتراضية؟
>
> - أ) أن تكون `static`
> - ب) أن تكون Constructor
> - ج) ✅ أن تكون Destructor
> - د) أن تُغيّر نوع الإرجاع في الكلاس المشتق 📎

> ما المؤشر `this` ؟
>
> - أ) مؤشر للكلاس الأساسي
> - ب) مؤشر للبرنامج الرئيسي `main`
> - ج) ✅ مؤشر خفي يشير للكائن الذي استدعى الدالة الحالية
> - د) مؤشر للذاكرة الديناميكية 📎

> كلاس `student` يحتوي على `virtual void display()` . مؤشر من نوع `student` يشير لكائن `academic` مشتق. ماذا يستدعي `ptr->display()` ؟
>
> - أ) دالة student دائماً
> - ب) ✅ دالة academic — لأن virtual تُفعّل Dynamic Binding
> - ج) يُعطي Compile Error
> - د) يعتمد على وقت التشغيل 📎

> "يمكن إنشاء مؤشر من نوع الكلاس المجرد رغم أنه لا يمكن إنشاء كائن منه"
>
> - أ) ✅ صحيح — المؤشر والمرجع مسموحان، فقط إنشاء الكائن ممنوع
> - ب) خاطئ 📎